

AI가 글을 읽는 법: 토큰에서 AI 에이전트까지

문장을 조각(토큰)으로 쪼개고, 화살표(벡터)로 바꾸고,
각도로 관련성을 재어 원문 근거로 답하는 RAG 에이전트 만들기

🕒 약 40분

📖 인터넷 되는 노트북

📖 선수 지식: 파이썬 변수·리스트·함수

💡 선생님과 함께 짚고 가는 기초 개념:

1. LLM (거대 언어 모델): 수억 개의 책과 글을 읽고 학습하여 인간처럼 말을 이해하고 다음 단어를 자연스럽게 이어 쓰는 거대한 언어 두뇌입니다.
2. AI 에이전트 (AI Agent): 단순한 챗봇을 넘어, '목표(Goal)'를 가지고 주변 문서나 환경을 '탐색(Search)'하고 '판단(Reason)'한 뒤, 스스로 '행동(Act)'하는 자율적인 인공지능입니다.
3. 토큰화 (Tokenization): 컴퓨터는 한글 글자를 그대로 이해하지 못합니다. 따라서 문장을 '의미 있는 작은 조각(토큰)'으로 쪼개고 각각 고유한 '숫자 번호표(ID)'를 부여해야 합니다.

🔥 실습 1: 문장을 잘게 쪼개보자! (토큰 쪼개기)

어절/단어 단위 토큰화

실습할 문장을 입력하거나 아래 프리셋을 클릭하세요:

어제까지도 너는 내게 한마디도 건네지 않더니, 오늘 점순이가 감자를 불쑥 내밀었다.

📖 《동백꽃》 문장 📖 《어린 왕자》 문장 📖 《학교생활규정》 조항 🗑️ 지우기

💡 컴퓨터가 인식한 토큰 및 고유 ID (Vocabulary)

총 11개 토큰 (고유 어휘 11단어)

각 단어 옆 파란 숫자는 컴퓨터가 단어를 기억하기 위해 붙인 고유 번호표(Token ID)입니다:

어제까지도	101	너는	102	내게	103	한마디도	104	건네지	105	않더니	106
오늘	107	점순이가	108	감자를	109	불쑥	110	내밀었다	111		

🔢 컴퓨터 내부의 숫자 변환 벡터 (ID 시퀀스):

[101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111]

▲ 문학 & 규정집 RAG AI 에이전트 실습실 — 토큰화부터 실시간 Q&A까지

START · 이 모듈의 지도

문장을 숫자로, 숫자를 화살표로, 화살표를 에이전트로

이 모듈에서는 AI가 글을 다루는 방식을 배웁니다. 컴퓨터는 한글을 그대로 이해하지 못합니다. 그래서 문장을 작은 조각(토큰)으로 쪼개 번호를 붙이고, 화살표(벡터)로 바꾼 뒤, 각도(코사인 유사도)로 관련성을 잹니다. 마지막에는 이 수학으로 원문 근거를 찾아 답하는 AI 에이전트를 직접 조립합니다.



학습 순서 — 4단계

1

1단계 · 토큰화

문장을 잘게 쪼개 고유 번호표(ID)를 붙입니다.

2

2단계 · 벡터와 코사인 유사도

문장을 화살표로 바꾸고 사잇각으로 관련성을 잹니다.

3

3단계 · AI 에이전트 만들기

데이터(책) + AI 두뇌(LLM)를 조립합니다.

4

4단계 · 실시간 Q&A

원문 근거를 찾아 답하는 에이전트와 대화합니다.



이 모듈에서 배우는 것 (3줄 요약)

- ① **문장을 숫자로**: 컴퓨터가 글을 작은 조각(토큰)으로 쪼개고 고유 번호표(ID)를 붙이는 원리
- ② **문장을 화살표로**: 문장을 좌표(벡터)로 바꾸고 사잇각의 코사인으로 관련성을 재는 수학
- ③ **나만의 AI 에이전트**: RAG(책에서 근거를 먼저 검색한 뒤 그 근거로 답을 생성)로 원문 근거를 찾아 답하는 에이전트 만들기



준비 완료 체크

- 인터넷이 되는 노트북을 준비했다.
- 선택: Google AI Studio API 키 (없어도 마지막 단계 전까지 모두 체험 가능)

도입 · 질문

컴퓨터는 한글을 읽을 수 없습니다

여러분이 읽는 "동백꽃" — 컴퓨터에게는 0과 1의 나열일 뿐입니다

사람은 글자를 보고 바로 뜻을 떠올리지만, 컴퓨터에게 한글 글자는 그냥 전기 신호의 패턴입니다. 그래서 AI가 글을 다루려면 **번역 과정**이 필요합니다. 글 → 작은 조각(토큰) → 번호(ID) → 화살표(벡터). 이 번역의 각 단계를 오늘 직접 체험합니다.

💡 기초 개념

LLM (거대 언어 모델)

수억 개의 글을 읽고 학습한 **거대한 언어 두뇌**입니다.

AI 에이전트

'목표'를 가지고 문서를 '탐색'하고 '판단'한 뒤 스스로 '행동'하는 인공지능입니다.

토큰화

문장을 '**의미 있는 작은 조각(토큰)**'으로 쪼개 '**숫자 번호표(ID)**'를 부여하는 과정입니다.

💡 비유 — 외국어 사전

한글을 모르는 외국인에게 한국 소설을 주면, 먼저 단어들을 사전에서 찾아 번호를 매기겠죠. 토큰화도 똑같습니다. 컴퓨터가 글을 다루기 위한 '**컴퓨터용 사전 만들기**'가 첫 단계입니다.

맞춤 독서 교실 · 💡 데이터(문학작품) · 토큰화 · 코사인 유사도 · AI 에이전트(LLM)의 원리를 체험합니다.

● 오프라인 모의 실험 5

🎯 이 모듈에서 배우는 것 (3줄 요약)

1. 문장을 **숫자로**: 컴퓨터가 글을 작은 조각(토큰)으로 쪼개고 고유 번호표(ID)를 붙이는 원리
 2. 문장을 **화살표**: 문장을 좌표(벡터)로 바꾸고, 사잇각의 코사인(코사인 유사도)으로 관련성을 재는 수학
 3. 나만의 **AI 에이전트**: RAG(검색 증강 생성 — 책에서 근거를 먼저 검색한 뒤, 그 근거로 답을 생성하는 방식)으로 원문 근거를 찾아 답하는 에이전트 만들기
- 🔥 준비물: 인터넷 되는 노트북 · 선수 지식: 파이썬 변수·리스트·함수 기초 · 선택: Google AI Studio API 키 (없어도 마지막 단계 전까지 모두 체험 가능!)

▲ 모듈 4 도입 — 문학·규정집 RAG AI 에이전트 실습실 전체 구성

개념 ①

토큰화 — 문장을 잘게 쪼개 번호표 붙이기

시뮬레이터의 **1단계 탭**에서 문장을 입력하고 ⚡ 텍스트를 컴퓨터용 토큰으로 분해하기 버튼을 눌러 보세요.

1 문장 입력

직접 입력하거나 프리셋을 클릭합니다. (예: 《동백꽃》 문장, 《어린 왕자》 문장, 《학교생활규정》 조항)

2 분해 실행

어절/단어 단위로 문장이 토큰으로 쪼개집니다.

3 ID 확인

각 토큰 옆 파란 숫자가 컴퓨터가 붙인 **고유 번호표(Token ID)**입니다.

4 벡터 확인

 숫자 변환 벡터(ID 시퀀스)에 문장 전체의 ID 나열이 표시됩니다.

예시

문장: "점순이가 감자를 내밀었다"

→ 토큰: [점순이가] [감자를] [내밀었다]

→ ID: [12] [45] [78] (예시 — 실제 번호는 사전 순서에 따라 다릅니다)

이렇게 문장이 **숫자의 나열**로 바뀌면, 컴퓨터는 비로소 글을 '계산할 수 있는 대상'으로 다루게 됩니다. 다음 단계에서는 이 숫자들을 **화살표**로 바꿉니다.

✓ 1단계 체크포인트 ✓

- 문장을 토큰으로 쪼개고, 각 토큰에 고유 번호표(ID)를 붙일 수 있다.

🔧 실습 1: 문장을 잘게 쪼개보자! (토큰 쪼개기)

어절/단어 단위 토큰화

실습할 문장을 입력하거나 아래 프리셋을 클릭하세요:

어제까지도 너는 내게 한마디도 건네지 않더니, 오늘 점순이가 감자를 불쑥 내밀었다.

개념 ②

코사인 유사도 — 문장의 관련성을 '각도'로 재다

토큰화된 문장들을 이제 **화살표(벡터)**로 바꿉니다. 문장에 나온 핵심 단어들의 빈도를 세어 2차원 공간 위의 화살표로 나타내는 거예요. 그리고 두 화살표 사이의 **사잇각**으로 두 문장이 얼마나 비슷한 주제인지 잹니다.

$$\text{유사도} = \cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$$

$A \cdot B$: 두 벡터의 내적 / $\|A\|$, $\|B\|$: 각 벡터의 길이

$$\cos(0^\circ) = 1.0$$

화살표가 같은 방향 → 완벽하게 같은 주제!

$$\cos(90^\circ) = 0.0$$

화살표가 직각 → 전혀 상관없는 판소리!

2D 의미 공간 속 화살표 각도기

시뮬레이터의 **2단계 탭**을 보세요. 질문 "점순이의 감자"(●)와 책 원문 "감자 세 개를 내밀었다"(●)의 사잇각 θ 가 표시됩니다. 각도가 좁을수록(0° 에 가까울수록) $\cos(\theta)$ 가 1에 가까워지고, 에이전트는 그 문장을 '**정답 근거**'로 선택합니다.

코사인 유사도 실시간 랭킹 검색기

질문을 입력하고  **유사도 계산하기**를 누르면, 후보 문장들이 유사도 순으로 정렬됩니다. 검색 엔진의 핵심이 바로 이 '관련성 점수 매기기'입니다.

2단계 체크포인트

- "두 문장의 사잇각이 좁을수록(0° 에 가까울수록) 같은 주제!"라고 설명할 수 있다.

개념 ③

RAG — 책에서 근거를 먼저 찾고 답하기

RAG(검색 증강 생성)는 '책에서 근거를 먼저 검색한 뒤, 그 근거로 답을 생성'하는 방식입니다. AI가 지어 내지 않고 원문을 근거로 답하게 만드는 핵심 기술입니다.



에이전트의 두 기둥



[기둥 1] 데이터

에이전트의 기억이자 책. 소설 원문, 학교 규정집, 내가 쓴 글 등.



경고: 데이터가 없으면 AI는 내용을 몰라 엉뚱한 상상(환각)을 지어냅니다!



[기둥 2] AI 두뇌

에이전트의 두뇌(LLM)이자 페르소나. Google Gemini 모델, 성격/말투, 판단 규칙.



경고: AI가 없으면 검색된 문장 조각만 덜렁 보여줄 뿐, 공감하고 설명하지 못합니다.



AI 에이전트 엄격 행동 지침 (환각 방지)

[원칙 1] 오직 등록된 [데이터 영역]의 본문 내용만을 근거로 삼아 답할 것

[원칙 2] 본문에 없는 내용은 추측하지 말고 "책에 나오지 않습니다"라고 솔직히 말할 것

[원칙 3] 답변 끝에 반드시 참고한 원문 구절과 번호를 근거로 표기할 것

이 세 원칙이 바로 AI의 거짓말(환각)을 막는 안전장치입니다. 여러분이 나중에 AI를 쓸 때도 '근거를 대라'고 요구하는 습관이 최고의 방어막이 됩니다.

체험 · 시뮬레이터

나만의 AI 에이전트 조립하기

시뮬레이터의 3단계 탭에서 [데이터 + AI 두뇌] 에이전트를 직접 조립합니다.

1 문서 선택

① 학습시킬 문서를 고릅니다: 🍪 《동백꽃》 🐼 《어린 왕자》 🏠 《학생생활규정》 ✎ 나만의 글

2 원문 편집

② 원문 텍스트 편집기에 소설/규정을 붙여넣고 ✂ 문장 청크로 쪼개어 저장합니다.

3 청크 확인

③ 분할 저장된 문장 조각(Chunks)이 쌓이는 것을 확인합니다.

4 페르소나 선택

① 캐릭터 프리셋을 고릅니다: 🍪 점순이 🦊 지혜로운 여우 ⚖ 선도부장/판사 ✨ 맞춤 캐릭터

5 완성

🚀 나만의 AI 에이전트 완성! → 4단계 탭으로 이동

💬 4단계 — 실시간 Q&A 대화

질문을 입력하면 [1] 📚 데이터 영역에서 코사인 유사도로 가장 관련 있는 문장을 먼저 찾아내고, [2] 🧠 AI 두뇌가 그 문장을 근거로 캐릭터 성격에 맞춰 대답합니다. 답변과 함께 🔍 검색된 원문 근거와 📊 유사도 Top 3, 🧠 프롬프트 조립 내역이 그대로 공개됩니다.

👉 교사용 팁 — 독서 교육

에이전트가 찾은 원문 근거를 확인하면서 작가의 의도나 복잡한 규정의 원문을 직접 대조해 보는 비판적 사고력과 문해력을 함께 기를 수 있습니다.

AI Life 홈04 자연어처리문학과 규정집 RAG AI 에이전트 실습실오프라인 시뮬레이션 모드구글 API 설정수업 종료Colab

1. LLM & 에이전트 개념 & 토큰화2. 텍스트 벡터 & 코사인 유사도3. 나만의 AI 에이전트 만들기4. 실시간 Q&A 대화 & 독서 팁구

AI 맞춤 독서 교실3단계: [데이터]와 [AI 두뇌]를 구분하고, 나만의 맞춤형 에이전트를 조립합니다.오프라인 모의 실행 모드

[기동 1] 데이터 (Data / 지식 베이스)

- 역할: 에이전트의 기억(Memory)이자 책
- 내용: 소설 원문, 학교 규정집, 내가 쓴 글 등
- 경고: 데이터가 없으면 AI는 소설 속 내용을 알지 못해 엉뚱한 상상(환각)을 지어냅니다!

RAG 연결

[기동 2] AI (Brain / 추론 엔진)

- 역할: 에이전트의 두뇌(LLM)이자 페르소나
- 내용: Google Gemini 모델, 성격/말투, 판단 규칙
- 경고: 시가 없으면 검색된 문장 조각만 덜렁 보여줄 뿐, 사람처럼 공감하고 맥락 있게 설명하지 못합니다!

1. 내 데이터(Data) 영역 (지식 베이스)점순이 문학 독서 에이전트

- ① 학습시킬 문서 선택 또는 직접 입력:
 - 《동백꽃》 (문학)
 - 《어린 왕자》 (문학)
 - 《학생생활규정》 (규정집)
 - 나만의 글 직접 입력

2. 내 AI 두뇌(Brain) 영역 (페르소나 & 규칙)내장 규칙 추론 엔진 (무료/오프라인)

- Google Gemini API Key (선택):
 - AltaSy... (입력 시 실제 Gemini가 페르소나 연거)
 - 연결 확인
- 키가 없어도 내장 오프라인 규칙 엔진으로 100% 작동합니다.
- 키 삭제

② 원문 텍스트 편집기 (소설/규정 붙여넣기):문장 청크로 쪼개어 저장외부 강의용 교재

▲ AI 에이전트 실습실 — 데이터와 두뇌를 조립하고 실시간으로 대화합니다

코드 · 파이썬

토큰화부터 RAG 검색까지 코드로

위에서 체험한 '토큰화 → 벡터 → 코사인 유사도 → RAG' 과정을 파이썬 코드로 그대로 옮긴 것입니다.

```
# 1. 문장 토큰화 및 어휘 사전(Vocab) 만들기
corpus = ["동백꽃이 붉게 피어나는 봄날이었다",
          "점순이는 나에게 구운 감자 세 개를 건넸다", ...]
vocab = sorted(list(set(" ".join(corpus).split())))
word_to_id = {w: i for i, w in enumerate(vocab)}

# 2. Bag-of-Words 문장 벡터화: 각 단어가 몇 번 나왔는지 세기
def text_to_vector(text):
    vec = np.zeros(len(vocab))
    for word in text.split():
        if word in word_to_id:
            vec[word_to_id[word]] += 1
    return vec

# 3. 코사인 유사도:  $\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$ 
def cosine_similarity(v1, v2):
    norm1, norm2 = np.linalg.norm(v1), np.linalg.norm(v2)
    if norm1 == 0 or norm2 == 0: return 0.0
    return float(np.dot(v1, v2) / (norm1 * norm2))

# 4. 질문에 가장 관련 깊은 근거 문장 검색 (RAG 검색기)
query = "점순이가 건넨 선물은 무엇인가?"
q_vec = text_to_vector(query)
scores = [cosine_similarity(q_vec, text_to_vector(doc)) for doc in corpus]
best = int(np.argmax(scores)) # 점수가 가장 높은 문장!
```

 (실습)_텍스트_에이전트.ipynb — 핵심 셀 (발췌)

 코드 읽기 포인트

- ① **word_to_id**: 시뮬레이터 1단계의 '파란 번호표'와 같은 것입니다.
- ② **text_to_vector**: 문장을 '단어 빈도 화살표'로 바꾸는 함수 — 2단계의 화살표 그리기입니다.
- ③ **cosine_similarity**: 개념 ②의 공식을 그대로 코드로 옮긴 것입니다.
- ④ **np.argmax(scores)**: 유사도 1등 문장을 '정답 근거'로 뽑는 RAG 검색기의 심장입니다.

실전 · 교실

프롬프트 조립과 교사용 API 설정

검색된 근거 문장은 **프롬프트(지시문)**로 조립되어 AI 두뇌에게 전달됩니다.

AI 두뇌에게 전달되는 조립된 프롬프트

당신은 소설 '동백꽃'의 '점순이'입니다.
[참고할 원문 근거]: "점순이는 나에게 구운 감자 세 개를 건넸다"
[질문]: 점순이가 건넨 선물은 무엇인가?
점순이의 톡톡 쓰는 새침데기 말투로 위 근거에 기반하여 대답해주세요.

조립된 프롬프트 예시

근거 + 질문 + 말투(페르소나)가 하나로 합쳐져 전달됩니다. 에이전트의 답변이 '점순이 말투'이면서도 '원문 근거'를 벗어나지 않는 비결이 바로 이 조립 과정입니다.

교사용 Google Gemini API 설정

선생님의 **Google AI Studio API 키**를 등록하면, 실제 최신 거대 언어 모델(Gemini)이 캐릭터 페르소나를 연기하며 생생하게 답변합니다. (수업 추천 모델: **gemini-1.5-flash** — 초고속 & 무료)

 **수업 보안 필수:** API 키는 브라우저를 닫으면 자동 삭제되는 임시 세션에만 보관됩니다. 수업이 끝난 후 [**수업 종료: API 키 즉시 파기**] 버튼을 눌러 브라우저 메모리에서 키를 즉시 소멸시키세요. 빔프로젝터 노출 방지를 위해 키는 기본적으로 마스킹(....) 처리됩니다.

교사용 팁 — 수업 운영

API 키 없이도 내장 오프라인 규칙 엔진으로 100% 작동합니다. 키 발급이 어려운 환경에서는 오프라인 모드로도 충분히 수업이 가능합니다.

Q&A

자주 묻는 질문 — 한 걸음 더 깊이

? 토큰은 왜 '단어'가 아니라 '조각'인가요?

한국어는 조사가 붙고(점순이 '가'), 신조어가 계속 생기기 때문에 단어로 딱 잘라 쪼개기 어렵습니다. 그래서 AI는 '의미 있는 작은 조각' 단위로 쪼갭니다. 예를 들어 "감자를"이 하나의 토큰이 될 수도 있고, "감자"+"를"로 나눌 수도 있습니다. 조각 단위가 유연할수록 모르는 단어도 부분 부분으로 이해할 수 있습니다.

? 코사인 유사도는 '뜻'을 정말 이해하는 건가요?

엄밀히 말하면 아닙니다. 이 모듈의 Bag-of-Words 방식은 '같은 단어가 겹치는 정도'를 재는 것에 가깝습니다. "감자"라는 단어가 양쪽에 모두 나오니 비슷하다고 판단하는 거죠. 진짜 '뜻'의 유사함을 재는 것은 훨씬 정교한 임베딩 기술이 필요하지만, 출발점의 아이디어는 오늘 배운 것과 같습니다.

? RAG가 있으면 AI가 거짓말을 안 하나요?

훨씬 줄어들지만 0은 아닙니다. 원문에 없는 내용을 AI가 '그렇듯하게' 메우는 경우도 있습니다. 그래서 [원칙 2] "책에 나오지 않습니다"라고 솔직히 말하기와 [원칙 3] 근거 표기가 중요합니다. 여러분도 AI의 답변을 볼 때 **항상 근거를 요구하는 습관**을 들이세요.

? 챗봇과 AI 에이전트는 뭐가 다른가요?

챗봇은 물으면 답하는 '응답 기계'에 가깝습니다. 에이전트는 **목표**를 가지고 **탐색(문서 검색) → 판단(유사도 계산) → 행동(근거 기반 답변)**을 스스로 수행합니다. 오늘 만든 독서 에이전트가 바로 그 미니 버전입니다.

정리 · 체크리스트

여기까지 되면 — 모듈 4 완주!

토큰화부터 AI 에이전트까지, AI가 글을 읽는 원리를 전부 직접 체험했습니다.

 핵심 개념 체크

- 토큰화가 문장을 조각내고 고유 ID를 붙이는 과정임을 설명할 수 있다.
- 코사인 유사도가 두 문장 화살표의 사잇각으로 관련성을 재는 것임을 설명할 수 있다. ($\cos 0^\circ=1$, $\cos 90^\circ=0$)
- RAG가 '근거 검색 → 근거 기반 생성'임을 설명할 수 있다.
- 환각이 무엇이고, 3원칙(원문 근거·솔직함·근거 표기)으로 막는지 설명할 수 있다.

 체험·코드 체크

- 문장을 토큰으로 쪼개고 ID를 확인했다.
- 유사도 랭킹에서 1등 문장이 정답 근거로 뽑히는 것을 보았다.
- 데이터+두뇌 에이전트를 조립하고 질문에 원문 근거가 붙어 나오는 것을 확인했다.
- `text_to_vector` → `cosine_similarity` → `argmax` 흐름을 읽을 수 있다.

 한 문장 정리

자연어처리 = 글을 조각(토큰)으로 쪼개고, 화살표(벡터)로 바꾸고, 각도(코사인 유사도)로 관련성을 재어, 원문 근거로 답하는 것.

 다음 읽기

다음 모듈에서는 글자에서 소리로 넘어갑니다. 컴퓨터는 귀도 없는데 어떻게 사람의 말을 들을까요? — 음성인식 AI의 세계로 초대합니다.

 교사용 팁 — 과제 제안

내가 좋아하는 책(또는 학교생활규정)으로 나만의 에이전트를 만들어 보세요. 친구가 엉뚱한 질문을 했을 때 에이전트가 "책에 나오지 않습니다"라고 솔직히 말하는지 테스트해 보는 것도 재미있습니다!